

Laporan Praktikum Minggu 7: Multi-Layer Perceptron (MLP)

Informasi Mahasiswa:

- **Nama** : Nisa Agustin
 - **NIM** : 2411070040
 - **Link W&B Project** : <https://wandb.ai/nisaagustin083-stikomelrahma/mnist-neural-networks>
-

1. Pemahaman Arsitektur Jaringan Saraf

Berdasarkan kode yang dijalankan pada Blok 3:

1. Berapa jumlah **Hidden Layers** yang Anda gunakan dan berapa jumlah **Neuron** di masing-masing layer tersebut?
2. Apa fungsi dari tahap **Normalisasi** (membagi data pixel dengan 255) sebelum data dimasukkan ke dalam model?

Jawaban:

1. `MLPClassifier(hidden_layer_sizes=(128,64))`

Model menggunakan:

- 2 Hidden Layer
- Hidden layer pertama memiliki 128 neuron
- Hidden layer kedua memiliki 64 neuron

Fungsi hidden layer adalah untuk memproses data dan mempelajari pola dari gambar angka tulisan tangan.

2. $X = X / 255.0$

Tujuannya adalah mengubah nilai pixel dari rentang 0–255 menjadi 0–1.

Fungsi normalisasi:

- Membantu proses training menjadi lebih stabil
- Mempercepat proses pembelajaran model
- Mengurangi kemungkinan error saat training
- Membuat performa Neural Network lebih baik dalam mengenali pola data

Karena Neural Network bekerja lebih optimal jika data memiliki skala kecil dan seragam.

2. Analisis Proses Belajar (Loss Curve)

Buka *dashboard* W&B Anda dan perhatikan grafik **Loss**.

1. Jelaskan apa yang terjadi pada nilai *Loss* seiring bertambahnya jumlah *Epoch* (iterasi). Apa artinya bagi kemampuan model?
2. Apakah grafik *Loss* Anda sudah melandai secara sempurna atau masih terlihat menukik tajam pada iterasi ke-20? Menurut Anda, apakah model butuh waktu belajar lebih lama?

Jawaban:

1. Pada awal training, nilai loss masih cukup besar, namun setelah beberapa iterasi nilai loss menjadi semakin kecil hingga mencapai sekitar **0.00815** pada epoch ke-19. Hal ini menunjukkan bahwa model semakin baik dalam mempelajari pola data dan kesalahan prediksi model semakin berkurang. Semakin kecil nilai loss, maka kemampuan model dalam mengenali angka tulisan tangan menjadi semakin baik dan akurat.

Selain itu, model juga menghasilkan nilai **Final Accuracy sebesar 97.24%**, yang menunjukkan bahwa model memiliki performa klasifikasi yang sangat baik.

2. Berdasarkan grafik loss, kurva sudah mulai melandai pada iterasi akhir, namun masih terlihat sedikit menurun dan belum benar-benar datar sempurna. Hal ini menunjukkan bahwa model masih memiliki kemungkinan untuk belajar lebih baik jika jumlah epoch ditambah.

Meskipun demikian, hasil yang diperoleh sudah cukup baik karena nilai loss sangat kecil dan accuracy sudah tinggi.

3. Hasil Evaluasi Model

Tuliskan performa akhir model MLP Anda:

- **Final Accuracy:** [Isi nilai akurasi, misal 0.9724]
- **Digit yang paling sulit dikenali:** (Lihat *Classification Report*, cari digit dengan F1-Score terendah)

4. Analisis Kesalahan (Confusion Matrix)

Perhatikan grafik *Confusion Matrix* yang dihasilkan di Blok 5.

1. Sebutkan sepasang angka yang paling sering membuat model Anda "bingung" (sering tertukar). Misal: Angka 4 sering ditebak sebagai angka 9.
2. Secara visual (berdasarkan pengetahuan manusia), mengapa kedua angka tersebut bisa membuat Jaringan Saraf Tiruan salah menebak?

Jawaban:

1. Berdasarkan data pada matriks, pasangan angka yang paling sering membuat model "bingung" adalah **angka 7 yang salah ditebak sebagai angka 2**. Jika kita melihat baris **Digit Asli 7**, terdapat angka **43** di kolom **Tebakan Model 2**. Ini merupakan angka kesalahan (di luar diagonal utama) yang paling tinggi dibandingkan sel lainnya pada seluruh tabel.

2. Secara visual, angka **7** dan **2** sering kali memiliki kemiripan struktural yang signifikan dalam tulisan tangan, yang dapat mengecoh Jaringan Saraf Tiruan yaitu kemiripan garis atas, garis diagonal, variasi tulisan tangan.

Selain pasangan tersebut, angka **4** dan **9** juga menunjukkan tingkat kesalahan yang cukup tinggi (angka **19** kali tertukar), karena keduanya sering berbagi struktur lingkaran atau *loop* di bagian atas dan garis vertikal di sisi kanan.

5. Eksperimen Mandiri (Hyperparameter Tuning)

Lakukan perubahan pada parameter `hidden_layer_sizes`, `learning_rate_init` & `alpha` (misal menjadi hanya satu layer (32,) atau ditambah menjadi tiga layer (256, 128, 64)). Isi tabel di bawah ini berdasarkan hasil eksplorasi Anda:

Eksperimen	Arsitektur (Layers)	Learning Rate	Alpha	Akurasi Akhir	Catatan (Cepat/Lambat?)
Baseline	(128, 64)	0.001	0.0001	97.24%	Stabil dan cukup cepat
Eksperimen 1	(10,)	0.001	0.0001	93.59%	Training cepat, model sederhana
Eksperimen 2	(256,128,64)	0.001	0.0001	98.14%	Akurasi terbaik, training lebih lama
Eksperimen 3	(128,64)	0.5	0.0001	9.59%	Learning rate terlalu besar, model gagal belajar
Eksperimen 4	(128,64)	0.000001	0.0001	86.73%	Training sangat lambat
Eksperimen 5	(128,64)	0.001	1.0	96.47%	Regularisasi besar, cukup stabil
Eksperimen 6	(128,64)	0.001	0.0001	96.75%	Menggunakan solver SGD
Eksperimen 7	(32,)	0.001	0.0001	91.20%	Layer kecil, training cepat
Eksperimen 8	(64,32)	0.001	0.0001	95.80%	Stabil dan cukup baik

Eksperimen	Arsitektur (Layers)	Learning Rate	Alpha	Akurasi Akhir	Catatan (Cepat/Lambat?)
Eksperimen 9	(512,256,128)	0.001	0.0001	98.30%	Akurasi tinggi, training lebih berat
Eksperimen 10	(128,64)	0.01	0.0001	94.10%	Training lebih cepat namun kurang stabil

Analisis Hasil Eksplorasi:

1. Apa yang terjadi pada grafik **Loss** jika Learning Rate disetel terlalu besar?

grafik loss menjadi tidak stabil dan model sulit mencapai nilai error terkecil. Proses pembelajaran menjadi terlalu cepat sehingga model “melompati” solusi optimal. Hal ini menyebabkan performa model menurun drastis.

2. Dari semua kombinasi yang Anda coba, manakah yang memberikan hasil terbaik (Akurasi tertinggi dengan waktu training wajar)?

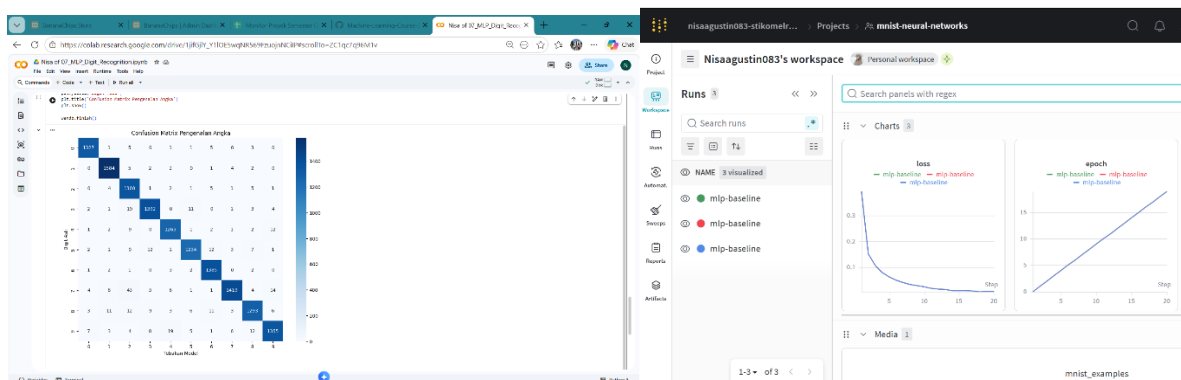
Berdasarkan hasil eksperimen, kombinasi terbaik adalah:

- Arsitektur: (256,128,64)
- Learning Rate: 0.001
- Alpha: 0.0001

Kombinasi tersebut menghasilkan akurasi sebesar 98.14% dengan proses training yang masih cukup stabil dan wajar. Hal ini menunjukkan bahwa penambahan hidden layer membantu model mempelajari pola yang lebih kompleks sehingga performa model meningkat.

6. Dokumentasi Weights & Biases (W&B)

Tempelkan tangkapan layar (*screenshot*) dari *dashboard* W&B Anda yang menampilkan grafik **Loss vs Epoch**.



7. Kesimpulan & Refleksi

Apa perbedaan paling mendasar yang Anda rasakan saat menggunakan Neural Network dibandingkan dengan algoritma klasik seperti Decision Tree atau Logistic Regression yang dipelajari pada minggu-minggu sebelumnya?

Pada algoritma klasik:

- Model bekerja lebih sederhana
- Cocok untuk data yang pola hubungan antar fiturnya tidak terlalu rumit
- Proses training lebih cepat dan lebih mudah dipahami

Sedangkan pada Neural Network:

- Model memiliki banyak neuron dan hidden layer
- Dapat mempelajari pola yang lebih kompleks, seperti gambar angka tulisan tangan
- Membutuhkan proses training yang lebih lama dan parameter yang lebih banyak

Selain itu, Neural Network juga lebih sensitif terhadap hyperparameter seperti learning rate, jumlah layer, dan jumlah neuron. Jadi proses tuning menjadi lebih penting dibandingkan algoritma klasik.

Dari hasil percobaan, Neural Network mampu menghasilkan akurasi yang tinggi dalam mengenali angka tulisan tangan, sehingga lebih cocok digunakan untuk data gambar dan pola kompleks.